SIMATS ENGINEERING

ASSESSMENT TOOL 2: Application Based Questions

COURSE NAME: Query Processing for Data Science **COURSE CODE:** DSA0501

COURSE OUTCOME COVERED

CO1: Explain the fundamentals of data wrangling, its importance, processes, and tools. Understand different data formats such as CSV, JSON, and XML along with machine-readable data.

Assessment Tool Weightage: 30%

Total Marks: 20

Student Name: Answar Vijes D **Register No: 192324028**

Code of Conduct

I Answar Vijes D / 192324028 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Rubrics:

Understanding of Problem Context (4 Marks)	Application of Concepts (4 Marks)	Problem-Solving Approach(4 Marks)	Accuracy of Solution (4 Marks)	Clarity (4 Marks)	Total (20 Marks)

Total Marks Awarded:

Signature of the Course Faculty

Answer Sheet - Application Based Questions

Course: Query Processing for Data Science (DSA0501) | Student: Answar Vijes D | Reg No: 192324028

Notes: All programs are written in Python 3. The question paper does not supply input data, so each answer uses small sample data created inside the program. The outputs shown were produced by actually running the code.

1. Student Admission Data Cleaning (10 Marks)

Task: A university has received a CSV file containing student admission records collected from multiple departments. The file contains duplicate records, missing phone numbers, inconsistent capitalization of department names, and extra spaces in student names. Write a Python program to read the CSV file, perform appropriate data wrangling operations to clean and standardize the data, remove duplicate records, and save the cleaned dataset into a new CSV file suitable for further analysis.

Approach: strip() removes extra spaces, title()/upper() fix capitalisation, missing phones are filled with 'Not Provided', drop_duplicates() removes repeated admissions and to_csv() saves the result.

Sample input (creates the files):

```
open("admissions.csv","w").write("""StudentID,Name,Department,Phone 1, Aarthi
Kumar ,cse,9876500001
2,Bharath Raj,ECE,
3,chandru S,Cse,9876500003
1, Aarthi Kumar ,CSE,9876500001
4, Deepa Lakshmi,mech,9876500004
5,Elango M,It,
5,Elango M,IT,
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("admissions.csv", dtype={"Phone": str})
print("Raw data:\n", df, "\n")

df["Name"] = df["Name"].str.strip().str.replace(r"\s+", " ", regex=True).str.title() df["Department"] =
df["Department"].str.strip().str.upper()
df["Phone"] = df["Phone"].fillna("Not Provided")

df = df.drop_duplicates().reset_index(drop=True)
df.to_csv("admissions_clean.csv", index=False)

print("Cleaned data (saved to admissions_clean.csv):\n", df) print("\nRecords: 7 -> %d"
% len(df))
```

Output:

```
Raw data:
   StudentID                Name  Department          Phone
0          1      Aarthi  Kumar         cse    9876500001
1          2       Bharath  Raj         ECE            NaN
2          3          chandru  S         Cse    9876500003
3          1      Aarthi  Kumar         CSE    9876500001
4          4       Deepa Lakshmi        mech    9876500004
5          5            Elango  M          It            NaN
6          5            Elango  M          IT            NaN

Cleaned data (saved to admissions_clean.csv):
   StudentID              Name Department          Phone
0          1    Aarthi Kumar         CSE     9876500001
1          2     Bharath Raj         ECE  Not Provided
2          3       Chandru S         CSE     9876500003
3          4  Deepa Lakshmi         MECH    9876500004
4          5         Elango M          IT  Not Provided

Records: 7 -> 5
```

2. Employee Data Integration (10 Marks)

Task: A company maintains employee details in a CSV file and department information in a JSON file. Both files contain a common Department ID field. Write a Python program to read the two files, combine the employee and department information, identify records with missing department details, and generate a consolidated dataset that can be stored in a relational database.

Approach: Read the CSV and JSON, left-merge on DeptID, flag rows where the department columns are NaN, and save a relational-ready CSV.

Sample input (creates the files):

```
import json
open("emp_int.csv","w").write("""EmpID,Name,DeptID,Salary
1,Arun,10,65000
2,Bhavana,20,48000
3,Charan,30,52000
4,Divya,10,72000
5,Elango,99,45000
""")
json.dump([{"DeptID":10,"DeptName":"IT","Location":"Chennai"},
           {"DeptID":20,"DeptName":"HR","Location":"Bengaluru"},
           {"DeptID":30,"DeptName":"Finance","Location":"Mumbai"}], open("dept_int.json","w"))
```

Program:

```
import pandas as pd

emp  = pd.read_csv("emp_int.csv")
dept = pd.read_json("dept_int.json")

merged = emp.merge(dept, on="DeptID", how="left")
missing = merged[merged["DeptName"].isnull()]
print("Records with missing department details:\n", missing, "\n")

merged["DeptName"]  =  merged["DeptName"].fillna("Unknown")
merged["Location"]  =  merged["Location"].fillna("Unknown")
merged.to_csv("employee_consolidated.csv", index=False)

print("Consolidated dataset (ready for database):\n", merged)
```

Output:

```
Records with missing department details:
   EmpID   Name  DeptID  Salary DeptName Location
4      5  Elango      99   45000      NaN      NaN

Consolidated dataset (ready for database):
   EmpID     Name  DeptID  Salary DeptName  Location
0      1     Arun      10   65000       IT   Chennai
1      2  Bhavana      20   48000       HR  Bengaluru
2      3   Charan      30   52000  Finance    Mumbai
3      4    Divya      10   72000       IT   Chennai
4      5   Elango      99   45000  Unknown   Unknown
```

3. Product Inventory Processing (10 Marks)

Task: An online retail company receives product information from suppliers in XML format. The XML file contains product names, categories, prices, and stock quantities. Write a Python program to parse the XML document, identify products with missing or invalid prices, correct the data wherever possible, and export the cleaned information into a CSV file for inventory management.

Approach: ElementTree parses the XML; pd.to_numeric(errors='coerce') exposes invalid prices; negative prices are made positive, and missing prices are filled with the category median before exporting to CSV.

Sample input (creates the files):

```
open("supplier.xml","w").write("""<?xml version="1.0"?>
<products>
  <product><name>Pen</name><category>Stationery</category><price>10</price><stock>200</stock></product>
<product><name>Notebook</name><category>Stationery</category><price>abc</price><stock>150</stock></product>
<product><name>Mouse</name><category>Electronics</category><price>450</price><stock>40</stock></product>
<product><name>Keyboard</name><category>Electronics</category><price></price><stock>25</stock></product>
<product><name>Pencil</name><category>Stationery</category><price>-5</price><stock>300</stock></product>
<product><name>Monitor</name><category>Electronics</category><price>7500</price><stock>12</stock></product>  </products>
""")
```

Program:

```
import pandas as pd
import xml.etree.ElementTree as ET

root = ET.parse("supplier.xml").getroot()
df = pd.DataFrame([{c.tag: c.text for c in p} for p in root.findall("product")])

df["price"] = pd.to_numeric(df["price"], errors="coerce")          # invalid text -> NaN df["stock"] = pd.to_numeric(df["stock"],
errors="coerce")
print("Parsed data:\n", df, "\n")

bad = df[df["price"].isnull() | (df["price"] <= 0)]
print("Products with missing / invalid prices:\n", bad, "\n")

df.loc[df["price"] < 0, "price"] = df["price"].abs()                    # fix sign errors df["price"] =
df["price"].fillna(df.groupby("category")["price"].transform("median"))

df.to_csv("inventory_clean.csv", index=False)
print("Cleaned inventory (saved to inventory_clean.csv):\n", df)
```

Output:

```
Parsed data:
          name        category     price    stock
0          Pen     Stationery       10.0      200
1     Notebook     Stationery        NaN      150
2        Mouse    Electronics     450.0       40
3     Keyboard    Electronics        NaN       25
4       Pencil     Stationery      -5.0      300
5      Monitor    Electronics    7500.0       12

Products with missing / invalid prices:
          name        category    price    stock
1     Notebook     Stationery       NaN      150
3     Keyboard    Electronics       NaN       25
4       Pencil     Stationery     -5.0      300

Cleaned inventory (saved to inventory_clean.csv):
          name        category     price    stock
0          Pen     Stationery       10.0      200
1     Notebook     Stationery        7.5      150
2        Mouse    Electronics     450.0       40
3     Keyboard    Electronics    3975.0       25
4       Pencil     Stationery        5.0      300
5      Monitor    Electronics    7500.0       12
```

4. Hospital Patient Data Validation (10 Marks)

Task: A hospital stores patient information in a CSV file containing patient IDs, names, ages, blood groups, and contact numbers. Some records contain missing values, invalid ages, and duplicate patient IDs. Write a Python
program to read the dataset, identify and correct data quality issues, remove duplicate records, and generate a validated dataset that can be imported into the hospital database.

Approach: Duplicate IDs are removed (first kept), missing values are filled (age by median, others by labels), invalid ages (outside 0-120) are replaced by the median age, and a validated CSV is produced.

Sample input (creates the files):

```
open("patients_raw.csv","w").write("""PatientID,Name,Age,BloodGroup,Contact  P01,Kamala,67,O+,9876500001
P02,Rajesh,-4,A+,9876500002
P03,Sundar,72,,9876500003
P04,Priya,34,B+,
P02,Rajesh,45,A+,9876500002
P05,Leela,250,AB-,9876500005
P06,,29,O-,9876500006
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("patients_raw.csv", dtype={"Contact": str}) print("Raw data:\n", df,
"\n")

print("Missing values:\n", df.isnull().sum(), "\n")
print("Duplicate patient IDs:", df["PatientID"].duplicated().sum()) df =
df.drop_duplicates(subset="PatientID", keep="first")

valid_age = df["Age"].between(0, 120)
print("Invalid ages:", (~valid_age).sum())
df.loc[~valid_age, "Age"] = int(round(df.loc[valid_age, "Age"].median()))          # correct invalid ages

df["BloodGroup"] = df["BloodGroup"].fillna("Unknown")
df["Contact"] = df["Contact"].fillna("Not Provided")
df["Name"] = df["Name"].fillna("Unknown")
df["Age"] = df["Age"].astype(int)

df.to_csv("patients_validated.csv", index=False)
print("\nValidated dataset:\n", df.reset_index(drop=True))
```

Output:

```
Raw data:
   PatientID      Name   Age BloodGroup        Contact
0        P01    Kamala    67         O+     9876500001
1        P02    Rajesh    -4         A+     9876500002
2        P03    Sundar    72        NaN     9876500003
3        P04     Priya    34         B+            NaN
4        P02    Rajesh    45         A+     9876500002
5        P05     Leela   250        AB-     9876500005
6        P06       NaN    29         O-     9876500006

Missing values:
 PatientID      0
Name            1
Age             0
BloodGroup      1
Contact         1
dtype: int64

Duplicate patient IDs: 1
Invalid ages: 2

Validated dataset:
   PatientID      Name   Age BloodGroup        Contact
0        P01    Kamala    67         O+     9876500001
1        P02    Rajesh    50         A+     9876500002
2        P03    Sundar    72    Unknown     9876500003
3        P04     Priya    34         B+  Not Provided
4        P05     Leela    50        AB-     9876500005
5        P06   Unknown    29         O-     9876500006
```

5. Library Database Preparation (10 Marks)

Task: A library receives book details in JSON format from different publishers. The records include book ID, title, author, publisher, and year of publication. Before storing the information in the library database, the data
must conform to the database schema. Write a Python program to read the JSON file, validate the required fields,
detect missing or inconsistent information, and produce a cleaned dataset ready for database insertion.

Approach: Validate required fields and year range for every JSON record; failing records are reported with the reason, and the remaining ones form the clean, schema-conformant dataset.

Sample input (creates the files):

```
import json
json.dump([
  {"book_id":1,"title":"Python Basics","author":"R. Kumar","publisher":"TechPress","year":2019}, {"book_id":2,"title":"Data
Science","author":"","publisher":"DataHouse","year":2021}, {"book_id":3,"title":"SQL Guide","author":"M.
Nair","publisher":"DBPub","year":"20X5"}, {"book_id":4,"title":"Machine Learning","author":"A. Rao","publisher":"AIBooks","year":2023},
{"book_id":1,"title":"Python Basics","author":"R. Kumar","publisher":"TechPress","year":2019}, {"book_id":5,"title":"Deep
Learning","author":"K. Das","year":2024},
], open("books_in.json","w"))
```

Program:

```
import json
import pandas as pd

REQUIRED = ["book_id", "title", "author", "publisher", "year"]
df = pd.DataFrame(json.load(open("books_in.json")))
df = df.reindex(columns=REQUIRED)                                    # enforce the schema columns print("Raw records:", len(df))

df = df.drop_duplicates(subset="book_id")
df["year"] = pd.to_numeric(df["year"], errors="coerce")

problems = []
for _, r in df.iterrows():
        why = [c for c in ["title", "author", "publisher"] if pd.isnull(r[c]) or str(r[c]).strip() == ""]          if pd.isnull(r["year"]) or not 1900 <=
r["year"] <= 2026:
                why.append("year")
        if why:
                problems.append((r["book_id"], why))

print("Inconsistent / missing information:")
for bid, why in problems:
        print(f"   book_id {bid}: invalid or missing -> {', '.join(why)}")

bad_ids = [b for b, _ in problems]
clean = df[~df["book_id"].isin(bad_ids)].copy()
clean["year"] = clean["year"].astype(int)
clean.to_csv("books_ready.csv", index=False)
print("\nDataset ready for database insertion:\n", clean)
```

Output:

```
Raw records: 6
Inconsistent / missing information:
  book_id 2: invalid or missing -> author
  book_id 3: invalid or missing -> year
  book_id 5: invalid or missing -> publisher

Dataset ready for database insertion:
   book_id              title      author  publisher  year
0        1      Python Basics   R. Kumar  TechPress  2019
3        4   Machine Learning     A. Rao     AIBooks  2023
```

6. Banking Transaction Analysis (10 Marks)

Task: A bank receives daily transaction details in CSV format from different branches. The dataset contains duplicate transaction IDs, blank account numbers, and inconsistent transaction types. Write a Python program to

import the CSV file, perform data wrangling operations to clean and standardize the records, identify invalid transactions, and save the processed data into a new file for further financial analysis.

Approach: Remove duplicate transaction IDs, standardise the transaction type text (Deposit / Withdrawal), flag blank account numbers and non-positive amounts as invalid, and save the processed file.

Sample input (creates the files):

```
open("transactions.csv","w").write("""TxnID,AccountNo,Type,Amount,Branch
T001,1001,deposit,5000,Chennai
T002,1002,WITHDRAW,2000,Madurai
T003,,Deposit,1500,Salem
T001,1001,deposit,5000,Chennai
T004,1003,Withdrawal,-300,Trichy
T005,1004,DEPOSIT ,8000,Chennai
T006,1005,withdraw,700,Madurai
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("transactions.csv")
print("Raw rows:", len(df))

df = df.drop_duplicates(subset="TxnID")
t = df["Type"].str.strip().str.lower()
df["Type"] = t.map(lambda x: "Deposit" if x.startswith("dep") else "Withdrawal")

invalid = df["AccountNo"].isnull() | (df["Amount"] <= 0)
print("\nInvalid transactions:\n", df[invalid], "\n")

valid = df[~invalid].copy()
valid["AccountNo"] = valid["AccountNo"].astype(int)
valid.to_csv("transactions_processed.csv", index=False)

print("Processed transactions:\n", valid, "\n")
print("Total by type:\n", valid.groupby("Type")["Amount"].sum())
```

Output:

```
Raw rows: 7

Invalid transactions:
   TxnID  AccountNo        Type  Amount  Branch
2  T003        NaN     Deposit    1500   Salem
4  T004     1003.0  Withdrawal    -300  Trichy

Processed transactions:
   TxnID  AccountNo        Type  Amount  Branch
0  T001       1001     Deposit    5000  Chennai
1  T002       1002  Withdrawal    2000  Madurai
5  T005       1004     Deposit    8000  Chennai
6  T006       1005  Withdrawal     700  Madurai

Total by type:
 Type
Deposit       13000
Withdrawal     2700
Name: Amount, dtype: int64
```

7. Online Shopping Order Processing (10 Marks)

Task: An e-commerce platform stores customer information in a JSON file and order details in a CSV file. Both datasets share a common Customer ID. Write a Python program to read the datasets, merge the customer and order
information, identify customers who have placed multiple orders, and generate a consolidated report in CSV format containing complete customer and order details.

Approach: Merge customers (JSON) with orders (CSV) on CustomerID, count orders per customer to find repeat buyers, and save the consolidated report.

Sample input (creates the files):

```
import json
json.dump([{"CustomerID":1,"Name":"Aarthi","City":"Chennai"},
           {"CustomerID":2,"Name":"Bharath","City":"Madurai"},
           {"CustomerID":3,"Name":"Chandru","City":"Salem"}], open("shop_customers.json","w"))
open("shop_orders.csv","w").write("""OrderID,CustomerID,Product,Amount
5001,1,Laptop,55000
5002,2,Mouse,450
5003,1,Keyboard,900
5004,3,Monitor,12500
5005,1,Headphones,1800
5006,3,Webcam,2100
""")
```

Program:

```
import pandas as pd

cust = pd.read_json("shop_customers.json")
orders = pd.read_csv("shop_orders.csv")

report = orders.merge(cust, on="CustomerID", how="left")
counts = report.groupby("Name")["OrderID"].count()
print("Orders per customer:\n", counts, "\n")
print("Customers with multiple orders:", counts[counts > 1].index.tolist(), "\n")

report["Order_Count"] = report.groupby("CustomerID")["OrderID"].transform("count")
report.to_csv("customer_order_report.csv", index=False)
print("Consolidated report:\n", report)
```

Output:

```
Orders per customer:
 Name
Aarthi    3
Bharath   1
Chandru   2
Name: OrderID, dtype: int64

Customers with multiple orders: ['Aarthi', 'Chandru']

Consolidated report:
   OrderID  CustomerID   Product  Amount     Name     City  Order_Count  0    5001
1     Laptop   55000    Aarthi  Chennai             3  1    5002             2     Mouse
450  Bharath  Madurai             1  2    5003          1    Keyboard    900   Aarthi  Chennai
3  3    5004          3    Monitor  12500  Chandru   Salem             2  4    5005
1  Headphones   1800    Aarthi  Chennai             3  5    5006          3     Webcam
2100  Chandru   Salem               2
```

8. University Course Information Management (10 Marks)

Task: A university maintains course details in an XML file and faculty information in a CSV file. The Course ID is common to both datasets. Write a Python program to parse the XML file, read the CSV file, integrate the two
datasets, identify courses without assigned faculty members, and generate a structured output suitable for updating
the university's academic database.

Approach: Parse the course XML, read the faculty CSV, left-merge on CourseID and find courses whose faculty columns are empty; save the structured output.

Sample input (creates the files):

```
open("courses.xml","w").write("""<?xml version="1.0"?>
<courses>
  <course><CourseID>C101</CourseID><Title>Data Structures</Title><Credits>4</Credits></course>
<course><CourseID>C102</CourseID><Title>Database Systems</Title><Credits>3</Credits></course>
<course><CourseID>C103</CourseID><Title>Machine Learning</Title><Credits>4</Credits></course>
<course><CourseID>C104</CourseID><Title>Cloud Computing</Title><Credits>3</Credits></course> </courses>
""")
open("faculty_c.csv","w").write("""FacultyID,FacultyName,CourseID
F1,Dr. Ramesh,C101
F2,Dr. Sulochana,C102
F3,Dr. Karthik,C103
""")
```

Program:

```
import pandas as pd
import xml.etree.ElementTree as ET

root = ET.parse("courses.xml").getroot()
courses = pd.DataFrame([{c.tag: c.text for c in x} for x in root.findall("course")]) faculty = pd.read_csv("faculty_c.csv")

merged = courses.merge(faculty, on="CourseID", how="left")
unassigned = merged[merged["FacultyID"].isnull()]
print("Courses without assigned faculty:\n", unassigned[["CourseID", "Title"]], "\n")

merged["FacultyName"] = merged["FacultyName"].fillna("Not Assigned") merged["FacultyID"] =
merged["FacultyID"].fillna("-")
merged.to_csv("course_faculty_output.csv", index=False)
print("Structured output:\n", merged)
```

Output:

```
Courses without assigned faculty:
      CourseID                Title
3        C104   Cloud Computing

Structured output:
      CourseID                 Title  Credits  FacultyID      FacultyName  0        C101     Data
Structures           4            F1       Dr. Ramesh  1        C102   Database Systems           3
F2   Dr. Sulochana  2        C103   Machine Learning           4            F3      Dr. Karthik  3
C104     Cloud Computing           3            -     Not Assigned
```

9. Vehicle Registration Data Processing (10 Marks)

Task: A transport department maintains vehicle registration records in CSV format, while owner information is available in JSON format. Some records contain duplicate registration numbers, missing owner details, and inconsistent date formats. Write a Python program to read both datasets, clean and standardize the information, merge the records using the registration number, and generate a validated dataset that follows the specified database schema.

Approach: Remove duplicate registration numbers, standardise mixed date formats with to_datetime(format='mixed'), merge owners (JSON) on RegNo, and flag records with missing owner details.

Sample input (creates the files):

```
import json
open("vehicles.csv","w").write("""RegNo,Type,RegDate
TN01AB1234,Car,15/03/2022
TN09CD5678,Bike,2023-07-21
TN22EF9012,Truck,Nov 2 2021
TN01AB1234,Car,15/03/2022
TN33GH3456,Car,2024-01-30
""")
json.dump([{"RegNo":"TN01AB1234","Owner":"Arun","Phone":"9876500001"},
           {"RegNo":"TN09CD5678","Owner":"Bhavana","Phone":"9876500002"},
           {"RegNo":"TN22EF9012","Owner":None,"Phone":"9876500003"}], open("owners.json","w"))
```

Program:

```
import pandas as pd

veh = pd.read_csv("vehicles.csv").drop_duplicates(subset="RegNo") own =
pd.read_json("owners.json", dtype={"Phone": str})

veh["RegDate"] = pd.to_datetime(veh["RegDate"], format="mixed", dayfirst=True).dt.strftime("%Y-%m-%d") merged = veh.merge(own,
on="RegNo", how="left")

incomplete = merged["Owner"].isnull() | merged["Phone"].isnull()
print("Records with missing owner details:\n", merged[incomplete], "\n")

valid = merged[~incomplete]
valid.to_csv("vehicles_validated.csv", index=False)
print("Validated dataset (schema: RegNo, Type, RegDate, Owner, Phone):\n", valid)
```

Output:

```
Records with missing owner details:
      RegNo    Type     RegDate  Owner       Phone
2  TN22EF9012   Truck   2021-11-02    NaN   9876500003
3  TN33GH3456     Car   2024-01-30    NaN          NaN

Validated dataset (schema: RegNo, Type, RegDate, Owner, Phone):       RegNo
Type       RegDate      Owner          Phone
0  TN01AB1234    Car   2022-03-15       Arun   9876500001
1  TN09CD5678   Bike   2023-07-21    Bhavana   9876500002
```

10. Retail Sales Data Transformation (10 Marks)

Task: A retail chain receives monthly sales information from different branches in CSV format, product details in JSON format, and supplier information in XML format. Write a Python program to import these machine-readable datasets, perform appropriate data wrangling operations, integrate the information using common fields, remove inconsistent and duplicate records, and generate a final consolidated dataset that can be directly loaded into a relational database for business reporting.

Approach: Import CSV (sales), JSON (products) and XML (suppliers); join them on ProductID and SupplierID, remove duplicate/inconsistent rows (unknown products, negative quantities) and write a final database-ready table.

Sample input (creates the files):

```
import json
open("branch_sales.csv","w").write("""SaleID,Branch,ProductID,Quantity,Month
1,Chennai,P1,10,Jan
2,Madurai,P2,5,Jan
3,Chennai,P3,8,Feb
2,Madurai,P2,5,Jan
4,Salem,P9,3,Feb
5,Chennai,P1,-2,Feb
6,Salem,P2,7,Mar
""")
json.dump([{"ProductID":"P1","ProductName":"Laptop","Price":55000,"SupplierID":"S1"},
                        {"ProductID":"P2","ProductName":"Mouse","Price":450,"SupplierID":"S2"},
                        {"ProductID":"P3","ProductName":"Monitor","Price":12500,"SupplierID":"S1"}], open("retail_products.json","w")) open("suppliers.xml","w").write("""<?xml
version="1.0"?>
<suppliers>
  <supplier><SupplierID>S1</SupplierID><SupplierName>TechSource</SupplierName></supplier>
  <supplier><SupplierID>S2</SupplierID><SupplierName>AccessoryHub</SupplierName></supplier>
</suppliers>
""")
```

Program:

```
import pandas as pd
import xml.etree.ElementTree as ET

sales = pd.read_csv("branch_sales.csv")
prod  = pd.read_json("retail_products.json")
sup   = pd.DataFrame([{c.tag: c.text for c in s}
                                        for s in ET.parse("suppliers.xml").getroot().findall("supplier")])

sales = sales.drop_duplicates()                                  # duplicate records sales = sales[sales["Quantity"] > 0]
# inconsistent quantities

final = (sales.merge(prod, on="ProductID", how="inner")   # unknown products dropped
.merge(sup, on="SupplierID", how="left"))
final["Revenue"] = final["Quantity"] * final["Price"]

final.to_csv("retail_final.csv", index=False)
print("Final consolidated dataset:\n", final.drop(columns="SupplierID")) print("\nTotal revenue:",
final["Revenue"].sum())
```

Output:

```
Final consolidated dataset:
   SaleID  Branch ProductID  Quantity  Month ProductName  Price  SupplierName  Revenue 0       1  Chennai
P1          10    Jan        Laptop  55000      TechSource   550000 1       2  Madurai          P2          5    Jan
Mouse     450  AccessoryHub      2250 2       3  Chennai          P3          8    Feb      Monitor  12500
TechSource   100000 3       6    Salem          P2          7    Mar        Mouse     450  AccessoryHub      3150

Total revenue: 655400
```

11. College Attendance Management (10 Marks)

Task: A college maintains daily attendance records in a CSV file. The file contains duplicate attendance entries, missing roll numbers, and inconsistent date formats. Write a Python program to read the file, clean the dataset,
remove duplicate records, standardize the dates, and generate a new CSV file that can be used for attendance analysis.

Approach: Drop rows with missing roll numbers, parse mixed date formats into one standard format, remove duplicate entries and save the cleaned attendance file.

Sample input (creates the files):

```
open("attendance_raw.csv","w").write("""RollNo,Name,Date,Status  101,Aarthi,01/07/2025,Present
102,Bharath,2025-07-01,Absent
,Chandru,2025-07-01,Present
101,Aarthi,2025-07-01,Present
103,Deepa,July  1  2025,Present
102,Bharath,02/07/2025,Present
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("attendance_raw.csv")
print("Raw rows:", len(df))

df = df.dropna(subset=["RollNo"])                                    # missing roll numbers
df["RollNo"] = df["RollNo"].astype(int)
df["Date"] = pd.to_datetime(df["Date"], format="mixed", dayfirst=True).dt.strftime("%Y-%m-%d") df = df.drop_duplicates(subset=["RollNo",
"Date"]).reset_index(drop=True)

df.to_csv("attendance_clean.csv", index=False)
print("Cleaned attendance (saved to attendance_clean.csv):\n", df)
```

Output:

```
Raw rows: 6
Cleaned attendance (saved to attendance_clean.csv):
    RollNo      Name        Date     Status
0     101    Aarthi   2025-07-01   Present
1     102   Bharath   2025-01-07    Absent
2     101    Aarthi   2025-01-07   Present
3     103     Deepa   2025-07-01   Present
4     102   Bharath   2025-07-02   Present
```

12. Electricity Consumption Analysis (10 Marks)

Task: An electricity board receives monthly power consumption data from different cities in CSV format. Some records contain invalid meter readings and missing consumer IDs. Write a Python program to identify incorrect
records, clean the dataset, and generate a summary showing the total electricity consumption for each city.

Approach: Identify invalid meter readings (negative / non-numeric / zero) and missing consumer IDs, remove them and use groupby('City') to total the consumption.

Sample input (creates the files):

```
open("power.csv","w").write("""ConsumerID,City,Month,Units  C001,Chennai,Jan,320
C002,Chennai,Jan,-45
,Madurai,Jan,210
C004,Madurai,Jan,180
C005,Salem,Jan,abc
C006,Salem,Jan,260
C007,Chennai,Jan,410
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("power.csv")
df["Units"] = pd.to_numeric(df["Units"], errors="coerce")

invalid = df["ConsumerID"].isnull() | df["Units"].isnull() | (df["Units"] <= 0) print("Incorrect records:\n",
df[invalid], "\n")

clean = df[~invalid]
summary = clean.groupby("City")["Units"].sum().reset_index(name="Total_Units")
summary.to_csv("city_consumption.csv", index=False)
print("Total electricity consumption by city:\n", summary)
```

Output:

```
Incorrect records:
      ConsumerID        City  Month    Units
1          C002    Chennai     Jan   -45.0
2           NaN    Madurai     Jan   210.0
4          C005      Salem     Jan      NaN

Total electricity consumption by city:
              City    Total_Units
0    Chennai             730.0
1    Madurai             180.0
2       Salem             260.0
```

13. School Examination Records (10 Marks)

Task: A school stores student information in a JSON file and examination marks in a CSV file. Write a Python program to read both files, combine the information using the student ID, calculate the total marks for each
student, and export the final report as a CSV file.

Approach: Read the student JSON and marks CSV, merge on StudentID, add up the subject marks to get Total, and export the final report.

Sample input (creates the files):

```
import json
json.dump([{"StudentID":1,"Name":"Aarthi","Class":"X-A"},
                        {"StudentID":2,"Name":"Bharath","Class":"X-A"},
                        {"StudentID":3,"Name":"Chandru","Class":"X-B"}], open("school_students.json","w"))
open("exam_marks.csv","w").write("""StudentID,Maths,Science,English
1,92,88,85
2,76,81,69
3,64,59,72
""")
```

Program:

```
import pandas as pd

students = pd.read_json("school_students.json")
marks = pd.read_csv("exam_marks.csv")

report = students.merge(marks, on="StudentID")
report["Total"] = report[["Maths", "Science", "English"]].sum(axis=1) report["Percentage"] =
(report["Total"] / 300 * 100).round(2)

report.to_csv("final_report.csv", index=False)
print(report)
```

Output:

```
   StudentID      Name  Class  Maths  Science  English  Total  Percentage  0          1    Aarthi
X-A         92           88           85       265           88.33  1                2    Bharath     X-A         76
81           69       226           75.33  2                3    Chandru      X-B         64           59           72
195           65.00
```

14. Weather Monitoring System (10 Marks)

Task: Weather stations provide daily weather observations in XML format. Write a Python program to parse the XML file, identify records containing missing temperature values, replace them with suitable values, and create a cleaned CSV file for further weather analysis.

Approach: Parse the XML observations; blank temperature values become NaN and are replaced by the city's average temperature (fallback: overall mean) before saving a clean CSV.

Sample input (creates the files):

```
open("weather.xml","w").write("""<?xml version="1.0"?>
<observations>
  <obs><city>Chennai</city><date>2025-07-01</date><temperature>34.5</temperature></obs>
<obs><city>Chennai</city><date>2025-07-02</date><temperature></temperature></obs>
<obs><city>Chennai</city><date>2025-07-03</date><temperature>33.5</temperature></obs>
<obs><city>Delhi</city><date>2025-07-01</date><temperature>38.0</temperature></obs>
<obs><city>Delhi</city><date>2025-07-02</date><temperature></temperature></obs>
<obs><city>Delhi</city><date>2025-07-03</date><temperature>40.0</temperature></obs>  </observations>
""")
```

Program:

```
import pandas as pd
import xml.etree.ElementTree as ET

root = ET.parse("weather.xml").getroot()
df = pd.DataFrame([{c.tag: c.text for c in o} for o in root.findall("obs")]) df["temperature"] =
pd.to_numeric(df["temperature"], errors="coerce")

print("Records with missing temperature:\n", df[df["temperature"].isnull()], "\n")

city_mean = df.groupby("city")["temperature"].transform("mean")
df["temperature"] = df["temperature"].fillna(city_mean).fillna(df["temperature"].mean())

df.to_csv("weather_clean.csv", index=False)
print("Cleaned weather data:\n", df)
```

Output:

```
Records with missing temperature:
     city        date  temperature
1 Chennai  2025-07-02          NaN
4   Delhi  2025-07-02          NaN

Cleaned weather data:
     city        date  temperature
0 Chennai  2025-07-01         34.5
1 Chennai  2025-07-02         34.0
2 Chennai  2025-07-03         33.5
3   Delhi  2025-07-01         38.0
4   Delhi  2025-07-02         39.0
5   Delhi  2025-07-03         40.0
```

15. Hotel Reservation Management (10 Marks)

Task: A hotel maintains booking details in CSV format and customer information in JSON format. Write a Python program to integrate the two datasets using the Customer ID, identify customers with multiple bookings,
and generate a consolidated reservation report.

Approach: Merge bookings (CSV) with customers (JSON) on CustomerID, count bookings per customer to find repeat guests and write the consolidated report.

Sample input (creates the files):

```
import json
open("bookings.csv","w").write("""BookingID,CustomerID,RoomType,CheckIn,Nights
B1,1,Deluxe,2025-08-01,2
B2,2,Standard,2025-08-03,1
B3,1,Suite,2025-08-10,3
B4,3,Deluxe,2025-08-12,2
B5,1,Standard,2025-08-20,1
B6,3,Suite,2025-08-25,4
""")
json.dump([{"CustomerID":1,"Name":"Aarthi","Phone":"9876500001"},
                    {"CustomerID":2,"Name":"Bharath","Phone":"9876500002"},
                    {"CustomerID":3,"Name":"Chandru","Phone":"9876500003"}], open("hotel_customers.json","w"))
```

Program:

```
import pandas as pd

book = pd.read_csv("bookings.csv")
cust = pd.read_json("hotel_customers.json")

report = book.merge(cust, on="CustomerID", how="left")
cnt = report.groupby("Name")["BookingID"].count()
print("Customers with multiple bookings:\n", cnt[cnt > 1], "\n")

report["Total_Bookings"] = report.groupby("CustomerID")["BookingID"].transform("count") report.to_csv("reservation_report.csv",
index=False)
print("Consolidated reservation report:\n", report)
```

Output:

```
Customers with multiple bookings:
  Name
Aarthi      3
Chandru     2
Name: BookingID, dtype: int64

Consolidated reservation report:
      BookingID  CustomerID  RoomType       CheckIn  Nights       Name          Phone  \ 0          B1
1      Deluxe  2025-08-01          2     Aarthi  9876500001  1          B2            2  Standard  2025-08-03
1  Bharath  9876500002  2          B3               1       Suite  2025-08-10          3     Aarthi  9876500001  3
B4               3      Deluxe  2025-08-12          2  Chandru  9876500003  4          B5               1  Standard
2025-08-20          1     Aarthi  9876500001  5          B6               3       Suite  2025-08-25          4  Chandru
9876500003

    Total_Bookings
0                3
1                1
2                3
3                2
4                3
5                2
```

16. Hospital Appointment Scheduling (10 Marks)

Task: A hospital stores doctor information in JSON format and appointment records in CSV format. Write a Python program to merge both datasets, identify appointments assigned to unavailable doctors, and generate a
report listing only valid appointments.

Approach: Doctors are stored in JSON with an 'available' flag; after merging with the appointments CSV, appointments whose doctor is unavailable or unknown are listed separately and only valid ones are written to the
report.

Sample input (creates the files):

```
import json
json.dump([{"DoctorID":1,"DoctorName":"Dr. Anand","Speciality":"Cardiology","available":True},
                    {"DoctorID":2,"DoctorName":"Dr. Meena","Speciality":"Pediatrics","available":False},
                    {"DoctorID":3,"DoctorName":"Dr. Rao","Speciality":"Orthopedics","available":True}], open("doctors.json","w"))
open("appts.csv","w").write("""AppID,Patient,DoctorID,Date
A1,Kamala,1,2025-07-01
A2,Baby Ria,2,2025-07-02
A3,Rajesh,3,2025-07-02
A4,Sundar,1,2025-07-03
A5,Leela,7,2025-07-03
""")
```

Program:

```
import pandas as pd

doctors = pd.read_json("doctors.json")
appts = pd.read_csv("appts.csv")

m = appts.merge(doctors, on="DoctorID", how="left")
unavailable = m[(m["available"] != True)]
print("Appointments with unavailable / unknown doctors:\n",
          unavailable[["AppID", "Patient", "DoctorID", "DoctorName"]], "\n")

valid = m[m["available"] == True].drop(columns="available")
valid.to_csv("valid_appointments.csv", index=False)
print("Valid appointments report:\n", valid)
```

Output:

```
Appointments with unavailable / unknown doctors:
   AppID   Patient  DoctorID  DoctorName
1    A2  Baby Ria         2   Dr. Meena
4    A5      Leela         7         NaN

Valid appointments report:
   AppID Patient  DoctorID        Date DoctorName    Speciality 0    A1  Kamala
1  2025-07-01  Dr. Anand    Cardiology 2    A3  Rajesh          3  2025-07-02
Dr. Rao  Orthopedics 3    A4  Sundar          1  2025-07-03  Dr. Anand
Cardiology
```

17. Online Examination System (10 Marks)

Task: An online examination portal stores student details in CSV format and answer submission logs in XML format. Write a Python program to process both datasets, identify students who have not submitted their answers,
and generate a report for the examination coordinator.

Approach: Read students (CSV) and submission logs (XML); a left merge shows students with no submission record, which are listed in the coordinator's report.

Sample input (creates the files):

```
open("exam_students.csv","w").write("""StudentID,Name,Email
1,Aarthi,aarthi@mail.com
2,Bharath,bharath@mail.com
3,Chandru,chandru@mail.com
4,Deepa,deepa@mail.com
5,Elango,elango@mail.com
""")
open("submissions.xml","w").write("""<?xml version="1.0"?>
<submissions>
  <submission><StudentID>1</StudentID><SubmittedAt>2025-09-01 10:55</SubmittedAt></submission>
<submission><StudentID>3</StudentID><SubmittedAt>2025-09-01 10:58</SubmittedAt></submission>
<submission><StudentID>4</StudentID><SubmittedAt>2025-09-01 11:00</SubmittedAt></submission> </submissions>
""")
```

Program:

```
import pandas as pd
import xml.etree.ElementTree as ET

students = pd.read_csv("exam_students.csv")
subs = pd.DataFrame([{c.tag: c.text for c in s}
                                    for s in ET.parse("submissions.xml").getroot().findall("submission")]) subs["StudentID"] =
subs["StudentID"].astype(int)

m = students.merge(subs, on="StudentID", how="left")
not_submitted = m[m["SubmittedAt"].isnull()][["StudentID", "Name", "Email"]]

not_submitted.to_csv("not_submitted_report.csv", index=False)
print("Students who have NOT submitted answers:\n", not_submitted)
print("\nSubmitted: %d | Not submitted: %d" % (m["SubmittedAt"].notnull().sum(), len(not_submitted)))
```

Output:

```
Students who have NOT submitted answers:
   StudentID     Name            Email
1          2  Bharath  bharath@mail.com
4          5   Elango   elango@mail.com

Submitted: 3 | Not submitted: 2
```

18. Railway Ticket Reservation (10 Marks)

Task: A railway reservation system stores passenger details in CSV format and train schedules in JSON format. Write a Python program to combine the two datasets, verify the train numbers for each passenger, and generate a final passenger reservation report.

Approach: Merge passengers (CSV) with train schedules (JSON) on TrainNo; passengers whose train number is not in the schedule are marked 'Invalid' and the final reservation report is saved.

Sample input (creates the files):

```
import json
open("passengers.csv","w").write("""PNR,Passenger,Age,TrainNo,Seat
1001,Aarthi,28,12601,S1-14
1002,Bharath,35,12627,S2-08
1003,Chandru,41,99999,S3-21
1004,Deepa,23,12601,S1-15
""")
json.dump([{"TrainNo":12601,"TrainName":"Chennai Mail","Departure":"22:00","From":"Chennai","To":"Bengaluru"},
{"TrainNo":12627,"TrainName":"Karnataka Express","Departure":"19:20","From":"Chennai","To":"Delhi"}], open("trains.json","w"))
```

Program:

```
import pandas as pd

pas = pd.read_csv("passengers.csv")
trn = pd.read_json("trains.json")

m = pas.merge(trn, on="TrainNo", how="left")
m["Train_Status"] = m["TrainName"].notnull().map({True: "Valid", False: "Invalid train number"})

m.to_csv("passenger_reservation_report.csv", index=False)
print(m[["PNR", "Passenger", "TrainNo", "TrainName", "Departure", "Train_Status"]])
```

Output:

```
    PNR Passenger  TrainNo     TrainName Departure  Train_Status
0  1001    Aarthi    12601  Chennai Mail     22:00         Valid
1  1002   Bharath    12627     Karnataka   19:20         Valid
2  1003   Chandru    99999           NaN       NaN  Invalid train number
3  1004     Deepa    12601  Chennai Mail     22:00         Valid
```

19. Insurance Claim Processing (10 Marks)

Task: An insurance company receives claim information in XML format and customer details in CSV format. Write a Python program to merge the datasets, identify claims without matching customer information, and generate a report of incomplete claims.

Approach: Parse claims from XML, merge with the customer CSV on CustomerID and report claims with no matching customer (or missing customer fields) as incomplete.

Sample input (creates the files):

```
open("claims.xml","w").write("""<?xml version="1.0"?>
<claims>
  <claim><ClaimID>CL1</ClaimID><CustomerID>1</CustomerID><Amount>25000</Amount></claim>
<claim><ClaimID>CL2</ClaimID><CustomerID>2</CustomerID><Amount>12000</Amount></claim>
<claim><ClaimID>CL3</ClaimID><CustomerID>9</CustomerID><Amount>40000</Amount></claim>
<claim><ClaimID>CL4</ClaimID><CustomerID>3</CustomerID><Amount>8000</Amount></claim> </claims>
""")
open("ins_customers.csv","w").write("""CustomerID,Name,PolicyNo
1,Aarthi,POL-101
2,Bharath,POL-102
3,Chandru,
""")
```

Program:

```
import pandas as pd
import xml.etree.ElementTree as ET

claims = pd.DataFrame([{c.tag: c.text for c in x}
                       for x in ET.parse("claims.xml").getroot().findall("claim")]) claims["CustomerID"]
= claims["CustomerID"].astype(int)
claims["Amount"] = claims["Amount"].astype(float)
cust = pd.read_csv("ins_customers.csv")

m = claims.merge(cust, on="CustomerID", how="left")
incomplete = m[m["Name"].isnull() | m["PolicyNo"].isnull()]

incomplete.to_csv("incomplete_claims.csv", index=False)
print("Report of incomplete claims:\n", incomplete)
```

Output:

```
Report of incomplete claims:
   ClaimID  CustomerID    Amount     Name PolicyNo
2      CL3           9  40000.0      NaN      NaN
3      CL4           3   8000.0  Chandru      NaN
```

20. Mobile Phone Sales Analysis (10 Marks)

Task: A mobile retailer stores product details in JSON format and sales transactions in CSV format. Write a Python program to combine the datasets, calculate the total quantity sold for each product, and save the results into
a new CSV file.

Approach: Merge the product JSON with the sales CSV, then groupby product name and sum Quantity; sort and save the totals.

Sample input (creates the files):

```
import json
json.dump([{"ProductID":1,"Model":"Galaxy S24","Brand":"Samsung"},
           {"ProductID":2,"Model":"iPhone 15","Brand":"Apple"},
           {"ProductID":3,"Model":"Redmi Note 13","Brand":"Xiaomi"}], open("phones.json","w"))
open("phone_sales.csv","w").write("""SaleID,ProductID,Quantity,Date
1,1,5,2025-06-01
2,2,3,2025-06-02
3,3,10,2025-06-02
4,1,4,2025-06-05
5,3,6,2025-06-07
6,2,2,2025-06-09
""")
```

Program:

```
import pandas as pd

prod = pd.read_json("phones.json")
sales = pd.read_csv("phone_sales.csv")

m = sales.merge(prod, on="ProductID")
total = (m.groupby(["Brand", "Model"])["Quantity"].sum()
.reset_index(name="Total_Quantity")
            .sort_values("Total_Quantity", ascending=False))

total.to_csv("phone_sales_summary.csv", index=False)
print(total.to_string(index=False))
```

Output:

```
  Brand          Model  Total_Quantity
 Xiaomi Redmi Note 13              16
Samsung      Galaxy S24               9
  Apple       iPhone 15               5
```

21. Employee Payroll Preparation (10 Marks)

Task: Employee information is stored in a CSV file, while salary details are maintained in a JSON file. Write a Python program to merge the datasets using Employee ID, identify employees with missing salary records, and prepare a payroll report.

Approach: Left-merge employees (CSV) with salary records (JSON); employees without salary data are listed, and the payroll (gross = basic + allowances - deductions) is prepared for the rest.

Sample input (creates the files):

```
import json
open("payroll_emp.csv","w").write("""EmpID,Name,Department
1,Arun,IT
2,Bhavana,HR
3,Charan,Finance
4,Divya,IT
""")
json.dump([{"EmpID":1,"Basic":50000,"Allowance":8000,"Deduction":3000},
           {"EmpID":2,"Basic":38000,"Allowance":5000,"Deduction":2000},
           {"EmpID":4,"Basic":60000,"Allowance":9000,"Deduction":4000}], open("salary.json","w"))
```

Program:

```
import pandas as pd

emp = pd.read_csv("payroll_emp.csv")
sal = pd.read_json("salary.json")

m = emp.merge(sal, on="EmpID", how="left")
missing = m[m["Basic"].isnull()]
print("Employees with missing salary records:\n", missing[["EmpID", "Name", "Department"]], "\n")

payroll = m.dropna(subset=["Basic"]).copy()
payroll["Net_Salary"] = payroll["Basic"] + payroll["Allowance"] - payroll["Deduction"] payroll.to_csv("payroll_report.csv",
index=False)
print("Payroll report:\n", payroll)
```

Output:

```
Employees with missing salary records:
   EmpID   Name  Department
2      3  Charan     Finance

Payroll report:
   EmpID    Name  Department    Basic  Allowance  Deduction  Net_Salary  0       1     Arun
IT  50000.0     8000.0      3000.0     55000.0  1       2  Bhavana          HR  38000.0
5000.0      2000.0     41000.0  3       4     Divya          IT  60000.0     9000.0     4000.0
65000.0
```

22. Movie Database Management (10 Marks)

Task: A movie streaming platform stores movie information in XML format and viewer ratings in CSV format. Write a Python program to combine the datasets, calculate the average rating for each movie, and generate a report sorted by rating.

Approach: Parse the movie XML, merge with ratings CSV on MovieID, take the mean rating per movie and sort in descending order.

Sample input (creates the files):

```
open("movies.xml","w").write("""<?xml version="1.0"?>
<movies>
  <movie><MovieID>1</MovieID><Title>Inception</Title><Genre>Sci-Fi</Genre></movie>
<movie><MovieID>2</MovieID><Title>Jai Bhim</Title><Genre>Drama</Genre></movie>
<movie><MovieID>3</MovieID><Title>Vikram</Title><Genre>Action</Genre></movie> </movies>
""")
open("ratings.csv","w").write("""ViewerID,MovieID,Rating
1,1,4.5
2,1,5.0
3,2,4.8
4,2,4.6
5,3,4.0
6,3,3.5
7,1,4.0
""")
```

Program:

```
import pandas as pd
import xml.etree.ElementTree as ET

movies = pd.DataFrame([{c.tag: c.text for c in m}
                       for m in ET.parse("movies.xml").getroot().findall("movie")]) movies["MovieID"] =
movies["MovieID"].astype(int)
ratings = pd.read_csv("ratings.csv")

m = ratings.merge(movies, on="MovieID")
report = (m.groupby(["Title", "Genre"])["Rating"].agg(Average_Rating="mean", Votes="count")
.round(2).reset_index().sort_values("Average_Rating", ascending=False))

report.to_csv("movie_rating_report.csv", index=False)
print(report.to_string(index=False))
```

Output:

```
      Title   Genre  Average_Rating  Votes
Jai Bhim   Drama            4.70      2
  Inception  Sci-Fi            4.50      3
   Vikram  Action            3.75      2
```

23. University Placement Records (10 Marks)

Task: Student details are available in a CSV file, while placement information is stored in JSON format. Write a Python program to integrate the datasets, identify students who have received placement offers, and prepare a
placement summary.

Approach: Merge student CSV with placement JSON on StudentID; students with a company in the placement record have received offers, and a summary by company is prepared.

Sample input (creates the files):

```
import json
open("placement_students.csv","w").write("""StudentID,Name,Department,CGPA
1,Aarthi,CSE,8.9
2,Bharath,ECE,7.8
3,Chandru,CSE,9.1
4,Deepa,IT,8.2
5,Elango,MECH,7.0
""")
json.dump([{"StudentID":1,"Company":"TCS","Package":6.5},
           {"StudentID":3,"Company":"Infosys","Package":7.2},
           {"StudentID":4,"Company":"TCS","Package":6.5}], open("placements.json","w"))
```

Program:

```
import pandas as pd

stu = pd.read_csv("placement_students.csv")
plc = pd.read_json("placements.json")

m = stu.merge(plc, on="StudentID", how="left")
placed = m[m["Company"].notnull()]
print("Students who received placement offers:\n", placed, "\n")

print("Placement summary")
print("Total students :", len(m))
print("Placed          :", len(placed), f"({len(placed)/len(m)*100:.0f}%)") print("Average package: %.2f
LPA" % placed["Package"].mean())
print("\nOffers by company:\n", placed["Company"].value_counts())
placed.to_csv("placement_summary.csv", index=False)
```

Output:

```
Students who received placement offers:
   StudentID     Name Department  CGPA  Company  Package
0          1   Aarthi        CSE   8.9      TCS      6.5
2          3  Chandru        CSE   9.1  Infosys      7.2
3          4    Deepa         IT   8.2      TCS      6.5

Placement summary
Total students : 5
Placed         : 3 (60%)
Average package: 6.73 LPA

Offers by company:
 Company
TCS        2
Infosys    1
Name: count, dtype: int64
```

24. Public Library Membership Analysis (10 Marks)

Task: A library stores member details in CSV format and book borrowing records in XML format. Write a Python program to identify members who have borrowed more than five books and generate a report containing their
details.

Approach: Parse the borrowing XML into a DataFrame, count books per member, keep members with more than five borrowings and merge with member details for the report.

Sample input (creates the files):

```
open("members.csv","w").write("""MemberID,Name,Phone
M1,Aarthi,9876500001
M2,Bharath,9876500002
M3,Chandru,9876500003
""")
rows = ["M1"]*7 + ["M2"]*3 + ["M3"]*6
xml = "".join("<borrow><MemberID>%s</MemberID><BookID>B%d</BookID></borrow>" % (m, i) for i, m in enumerate(rows, 1))
open("borrowings.xml","w").write('<?xml version="1.0"?><borrowings>%s</borrowings>' % xml)
```

Program:

```
import pandas as pd
import xml.etree.ElementTree as ET

members = pd.read_csv("members.csv")
borrow = pd.DataFrame([{c.tag: c.text for c in b}
                       for b in ET.parse("borrowings.xml").getroot().findall("borrow")])

counts = borrow.groupby("MemberID")["BookID"].count().reset_index(name="Books_Borrowed") heavy =
counts[counts["Books_Borrowed"] > 5].merge(members, on="MemberID")

heavy.to_csv("frequent_borrowers.csv", index=False)
print("Members who borrowed more than five books:\n", heavy)
```

Output:

```
Members who borrowed more than five books:
  MemberID  Books_Borrowed     Name       Phone
0       M1               7   Aarthi  9876500001
1       M3               6  Chandru  9876500003
```